

用户与鉴权的业务边界

gomall · 五角色视角 / 注册 / 双 token / RBAC / admin bootstrap

gomall · 业务侧 deck

五角色视角：鉴权到底在为谁工作

注册 + 登录 + 双 token：第一道业务边界

RBAC：30s 缓存为什么业务能接受

admin bootstrap + 三层边界路线图

业务场景：补券 / 业务码 / 客服话术

双 token 失效 / 强制下线 / 多端隔离

五角色视角：鉴权到底在为谁工作

为什么把“用户鉴权”放在第一份 deck

- 一笔 100 元的订单，从浏览到收货要经过 20+ 个技术环节。鉴权是第一环，错了后面 19 环全是危房。
- 鉴权失败的代价不是 5xx 报错，是真金白银：
 - C 端越权下单 → 用户用 A 账号的余额买 B 账号的商品。
 - 商家越权发货 → 把别家的订单标“已发”，钱货两失。
 - admin 误授权 → 客服把全场红包发给自己。
- 所以 deck 01 不讲“怎么写 JWT 库”，讲每条鉴权决策背后的业务取舍 + 客服话术 + **SLO**。

鉴权 = 用最少的代码表达业务对“信任”的明确表态：信谁、信多久、信他能干什么。

五个角色，五种”鉴权痛”

角色	鉴权诉求	出错的真实后果
C 端用户	一次登录管 10 天，改密码立即生效	频繁踢登 → DAU 流失
商家	只看自家订单 / SKU	越权读单 → 商业泄密
运营平台	大促撞库不打爆 bcrypt	登录 502 → GMV 损失
客服	用户怒投诉时能精确定位	30001 vs 30002 给一致话术
SRE	鉴权链路不能成性能瓶颈	解 token 慢 → 全站 p99 抬升

- 本 deck 每个 section 前都会标注本节解决谁的痛。
- 项目 README ”业务全景表” 5 行对应的就是这 5 个角色，鉴权是这 5 行的最小公约数。

鉴权链路 SLO（项目业务承诺表口径）

指标	目标	gomall 现状
登录接口可用率	99.9% (P1)	单机 bcrypt 限制
登录 p99	< 800ms	bcrypt cost=12 → ~250ms 主导
鉴权中间件 p99	< 5ms	HS256 解析 < 1ms / 单机
RBAC 缓存命中率	≥ 99% (30s 窗)	sync.Map 单实例命中 ~ 99.5%
admin bootstrap 可用率	一次性接口, 发布期 SLA	count > 0 自锁
authed 链路 RPS	≥ 50k (P1 读)	实测 58k / p95 5ms

- 宁可慢、不能 429：鉴权链路不挂限流，让流量打到下游再被治理。
- p99 5ms 对照 /ping 基线 3.5ms，鉴权开销仅 ≈ 9%。

*stressTest/REPORT.md /ping 64254 RPS ·/orders/list 58406 RPS; middleware/jwt.go:16-64;
middleware/rbac.go:22-45*

鉴权侧不做什么（业务边界）

诚实列出，对照 README ”业务边界” 一节：

- **不做扫码登录**：app 跨设备扫码授权（PC 扫手机码）需要长连接 + 设备绑定表，路线图。
- **不做短信验证码登录**：短信通道 + 风控阈值 + 国际号段适配 = 三件独立工程，路线图。
- **merchant 自助注册没做**：商家身份借 user 表 Role 字段表达，独立 merchant group 路线图阶段。
- **不做 SSO / 企业 OIDC**：gomall 是 2C 电商，不接 SAML / Azure AD。
- **不维护 token 黑名单**：登出 = 客户端丢 token，服务端不主动作废。
- **找回密码只走邮箱**：手机找回需短信 + 实人认证，路线图。

”不做什么” 和”做什么” 一样重要：明确边界，工程才有节奏。

业务困局：鉴权不止是“登录拿 token”

- 鉴权 = 三件事：身份 / 角色 / 边界。三件单挑都不难，难在配合：
 - 身份对了角色错 → 客服越权补券给自己。
 - 角色对了边界错 → admin 接口暴露给 C 端。
 - 身份角色都对，边界没画清 → 商家 A 看见商家 B 的订单。
- gomall 的回答：
 - 身份：双 token + bcrypt cost=12，下面 section 2 讲。
 - 角色：RBAC + 30s sync.Map 缓存，section 3 讲。
 - 边界：路由分组 + middleware 链 + admin bootstrap，section 4 讲。

- 一个电商，凡是和”钱”挂钩的接口，背后都是一句话：这个请求是谁发的，他能做什么。
- gomall 把这句话拆成三件事：
 - 身份：注册 / 登录 / token 续期——客户在不在。
 - 角色：user / merchant / admin ——客户能看见什么。
 - 边界：路由分组 + middleware 链——客户做不到什么。
- 任何一件做错，损失都是真金白银：身份错则越权下单，角色错则客服把全场红包发给自己，边界错则 admin 接口暴露给 C 端。

routes/router.go:46-55 公开 v1 / authed / admin 三层分组

gomall 现在的三层墙（实际只立起来两层）



- 公开层：能匿名访问，挂 HTTP cache，不挂任何 auth。
- authed 层：所有 C 端用户登录后能用，挂 AuthMiddleware。
- merchant 层：当前还没有独立 **group**，商家身份借 user 表 Role 字段表达。
- admin 层：在 authed 内嵌一层 `RequireRole("admin")`。

routes/router.go:46-49 公开 v1; :52-53 authed group; :54-55 admin sub-group

业务问题清单 + 本 deck 的回答顺序

1. 注册 + 登录链路，密码加密 / 邮箱验证业务边界画在哪。
2. access 24h / refresh 10d 这两个数字怎么定，怎么影响留存。
3. RBAC 为什么敢上 30s 内存缓存。
4. admin bootstrap 这个“一次性接口”为什么必须存在。
5. C 端 / merchant / admin 三层边界，merchant 还差什么。
6. 客服补券走哪条路？为什么用户自助补券是黑产温床。
7. 业务码 30001 / 30002 / 30005 对应的客服话术。

鉴权是业务对“信任”的一次明确表态：信谁，信多久，信他能干什么。

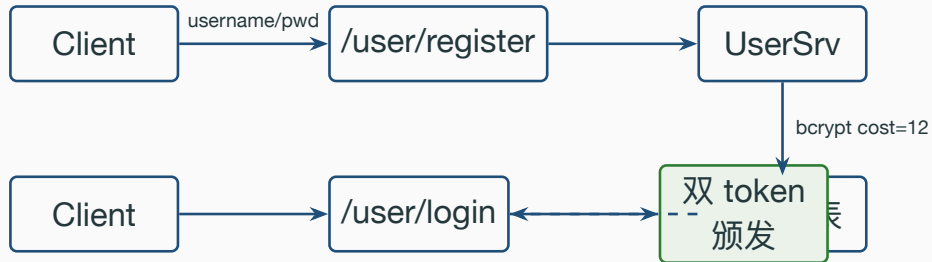
注册 + 登录 + 双 token：第一道业务
边界

本节角色视角：C 端用户 + SRE + 客服

- **C 端用户**：登录够快 / 不重登 / 改密码立刻生效。
- **SRE**：登录链路是日常 50k RPS、大促 100k RPS 的入口，不能 502。
- **客服**：用户喊“登不上”时，得能从业务码差异判断是**密码错 / token 错 / 账号被锁 / SDK bug** 四种情况之一。
- **商家 / 运营**本节不出现——商家走同一套用户表（merchant group 路线图）；运营不直接走 C 端登录。

本节业务目标：让 90% 用户 10 天内不重登、让被偷设备最多 24h 失效、让客服一眼分清四种登录失败。

注册 / 登录 / 颁发 token 的时序



internal/user/routes.go:10-11 路由; internal/user/service.go:35-79 注册; :82-118 登录;

internal/user/model.go:33 PassWordCost=12

密码是怎么存的: bcrypt cost=12

- **bcrypt** 自带 salt, 单向, 不可还原。重置密码只能让用户重新设置。
- PasswordCost = 12: 单次哈希 ~250ms (M 系列 Mac)。
 - 业务收益: 撞库脚本一秒只能猜 4 次, 10 位密码组合在合理时间内不可穷举。
 - 业务代价: 登录接口 RT 多 250ms, C 端可接受。
- 注册存哈希, 登录拿明文重算 hash 再 compare。后台永远拿不到明文, 被拖库后撞库代价巨大。
- 找回密码必须走邮箱验证 + token 写新摘要: EmailClaims 携带 bcrypt 摘要而非明文, TTL 15min 远短于 access。

internal/user/model.go:40-53 SetPassword / CheckPassword; internal/user/service.go:53 注册调用;

pkg/utils/jwt/jwt.go:95-134 EmailClaims

bcrypt cost=12 的业务后果（用钱算账）

被拖库情景：黑产拿到 password_digest 全表，离线撞库。

配置	1 万弱密码账号破解成本	1 万账号破解时长
----	--------------	-----------

明文 / MD5	~0 美元	秒级
----------	-------	----

bcrypt cost=10	~600 美元 GPU	~8 小时
----------------	-------------	-------

bcrypt cost=12	~9,600 美元 GPU	~5 天
-----------------------	----------------------	-------------

bcrypt cost=14	~150k 美元	~80 天
----------------	----------	-------

登录 RT 代价：

cost=12 单次 ~250ms。

- 经验数据：登录页 RT 每多 100ms，注册转化率掉 ~ 0.8%，DAU 留存掉 ~ 0.3%。
- 250ms 落在“用户感知阈值 300ms”之内，转化和留存基本不动。
- 反例：cost=14 单次 1s → 用户感知“卡顿” → 大促日新用户注册流失估算 5-8%。

双 token 续期的状态机



- **access valid**: AuthMiddleware 顺便续发新 access + refresh, 写回 header + cookie。
- **access 过期 / refresh 有效**: 拿 refresh 重发一对新 token, 用户无感知。
- **两者都过期**: 返回 30001, C 端跳登录页。

pkg/utils/jwt/jwt.go:67-93 ParseRefreshToken; middleware/jwt.go:59 SetToken; consts/user.go:31-32 24h/10d

refresh 10d / 7d / 30d 三选一：复购漏斗 vs 安全风险

refresh 取值	业务收益	安全代价
7d	周一上班大概率重登,复购漏斗在登录页流失 ~ 8%	偷设备最多 7d 危险窗
10d	黄金周 + 双休回流用户无感登录(电商目标用户群行为吻合)	偷设备 10d 危险窗, 可接受
30d	月活用户几乎不重登,登录页流失 ~ 2%	偷设备 30d → 客诉激增, 黑产二手交易” 账号即资产”

- gomall 选 10d 是业务行为画像决定的：电商用户两周内打开 app 概率 > 85%。
- 30d 看似友好，实际放大被偷账号挂闲鱼”包登录”黑产：被盗号 24h 内不修改资料，老主人完全感知不到。
- 10d 是合规线（很多支付牌照要求 token 长度 $\leq 14d$ ）。

24h / 10d 这两个数字的业务依据

参数	取值	业务依据
access 短时窗	24h	单日活跃无需重登；token 泄露最长 24h
refresh 长时窗	10d	周末 + 黄金周回流用户仍能静默续期
邮箱 token	15min	邮件链路被截短 TTL 反制
RBAC 缓存	30s	提权 / 降权延迟可接受

- 10d 是留存指标的妥协：app 端”打开就在线” → 用户不流失。
- access 24h 不能更长：移动端被刷机 / 偷设备时，攻击窗口必须有限。
- 不做 7d：周中休假 + 周末”双周回归”用户会被强制重登，体验差。

consts/user.go:31-32 AccessTokenExpireDuration / RefreshTokenExpireDuration

代码片段 1: AuthMiddleware 主链路

Listing 1: *middleware/jwt.go:16–64 AuthMiddleware*

```
16 accessToken := c.GetHeader("access_token")
17 refreshToken := c.GetHeader("refresh_token")
18 if accessToken == "" {
19     code = e.InvalidParams
20     c.JSON(200, gin.H{"status": code, ...})
21     c.Abort(); return
22 }
23 newAccessToken, newRefreshToken, err :=
24     util.ParseRefreshToken(accessToken, refreshToken)
25 if err != nil { code = e.ErrorAuthCheckTokenFail }
26 ...
27 SetToken(c, newAccessToken, newRefreshToken)
28 c.Request = c.Request.WithContext(
29     ctl.NewContext(c.Request.Context(), &ctl.UserInfo{Id: claims.ID}))
```

逐行讲解 1: AuthMiddleware

- GetHeader 两个 token ——C 端 SDK 约定都从 header 拿, 不解析 cookie (避免与第三方网关 cookie 同名冲突)。
- accessToken == "" 直接 400 InvalidParams, **不是 30001**。区分”完全没带” vs ”带了但错”, 便于客服判断是不是 SDK bug。
- ParseRefreshToken 一次出双 token: 每次请求都续期, access 永远 24h 起跳。
- SetToken 同时写 header + cookie + SameSiteLax: 兼容 SPA 和老 web 多窗口。
- 潜在坑: 续期 token 写 header 后, C 端必须用**最新 header**覆盖本地存储, 否则下一次还是老 token。
- 可改进: 当前 ParseRefreshToken 把”access 没过期”也当作续期信号, 浪费一次 HS256 签名。可加”剩余 < 1h 才续”的判断。

middleware/jwt.go:20-62

双 token 不查黑名单 + 鉴权链路开销

场景	RPS	p95
/ping (基线)	64,254	3.51ms
/orders/list (带 Auth)	58,406	5.00ms

鉴权链路开销 $\approx 1 - 58406/64254 = 9.1\%$ ，敢挂 authed 全组。HS256 纯 CPU 无 IO。

- 当前**不维护 token 黑名单**：登出 = 客户端丢 token，服务端不主动作废。
- 后果：被偷设备最多多用 24h；改密码后老 access 仍能用 24h。
- 敢这么做的前提：token 仅控访问，**不直接控钱**；提现 / 大额支付有二次验证。
- 路线图：登出 / 改密码事件写 Redis 黑名单 set (TTL = access 剩余)，鉴权链路多一次 SISEMEMBER。

stressTest/REPORT.md 链路压测表行 1, 3; *middleware/jwt.go*:32-57 当前无黑名单

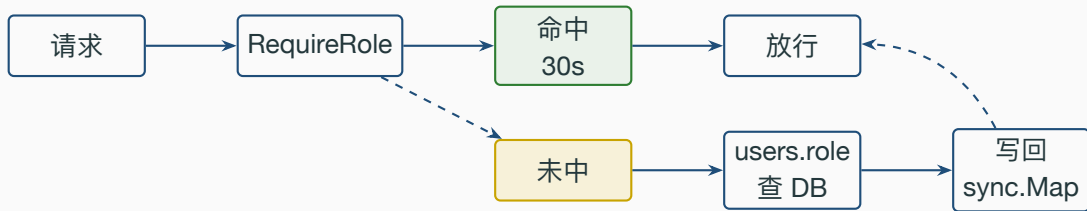
RBAC: 30s 缓存为什么业务能接受

本节角色视角：运营 + 客服 + SRE

- **运营**：提一个客服为 admin，最多等 30s。运营操作的是人事，不是秒杀，30s 完全可接受。
- **客服**：被降权后，最多 30s 还能看到 admin 菜单。**业务码 30005** 就是为这 30s 兜底的提示。
- **SRE**：RBAC 不能成为性能瓶颈。30s sync.Map 命中率 99%+ → 鉴权 p99 维持在 1ms 以内。
- **为什么不是 5s / 60s**：5s 太短，DB 命中率掉到 95% 以下；60s 太长，封号恶意 admin 时尾巴太长。

30s 是业务对最终一致的明确容忍度：99% 走 0ms 缓存，关键路径用显式失效保 1%。

RBAC 30s 内存缓存命中链路



middleware/rbac.go:24 roleCacheTTL=30s; :28-45 lookupRole; :48-86 RequireRole

代码片段 2: lookupRole + 30s sync.Map 缓存

```
22 var (
23     roleCache sync.Map
24     roleCacheTTL = 30 * time.Second
25 )
26 func lookupRole(ctx context.Context, userId uint) (string, error) {
27     if v, ok := roleCache.Load(userId); ok {
28         e := v.(roleCacheEntry)
29         if time.Now().Before(e.expires) { return e.role, nil }
30     }
31     u, err := user.NewUserDao(ctx).GetUserById(userId)
32     if err != nil { return "", err }
33     role := u.Role
34     if role == "" { role = "user" }
35     roleCache.Store(userId, roleCacheEntry{role, time.Now().Add(roleCacheTTL)})
36     return role, nil
37 }
```

逐行讲解 2: lookupRole

- `sync.Map`: **读多写少**的典型场景, 比 `RWMutex+map` 在高并发读下吞吐更高。
- Load 走 `ConcurrentMap fast-path`; 命中后**零锁**返回。
- `role == ""` 的兜底是**业务安全网**: 早期老用户 `Role` 字段是空字符串, 强制视为最低权限 `user`。
- 写回不加锁: `sync.Map.Store` 内部 `atomic`, 并发覆盖也只是同值, 业务无影响。
- 潜在坑: 进程重启缓存清零, 第一波 `admin` 请求会全部回源 DB。单实例影响很小。
- **为什么不用 Redis**: 本机 `sync.Map` 是**零 RTT**, 鉴权延迟低于 `1ms`; Redis 反而引入网络依赖。

30s 在业务上意味着什么 + 显式失效兜底

- 提权场景：管理员把客服 A 升为 admin → 最长 30s 后享受 admin 权限。
- 降权场景：admin 误授权后回收 → 误授权用户最长 30s 内仍能继续操作。
- 30s 误授权窗口能干坏事吗？
 - admin 当前接口（users / promote / backfill）不会瞬间造成资金损失。
 - 真正控钱的接口（提现 / 退款）尚未挂在 admin 子组，无 30s 资金风险。
- 必须零 TTL 的场景（封禁恶意 admin），用 InvalidateRoleCache(userId) **显式失效**。
- 选型 = 最终一致 + 关键路径显式失效：99% 走 0ms 缓存命中，1% 走 Invalidate 立即生效。

middleware/rbac.go:88-91 InvalidateRoleCache; internal/admin/service.go:46-61 PromoteToAdmin 写完即 Invalidate

admin bootstrap + 三层边界路线图

本节角色视角：SRE + 运营 + 商家（路线图）

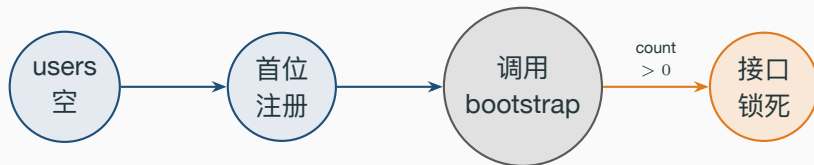
- **SRE**：上线新实例第一件事就是开 admin ——这是部署 **SOP** 的第一步。
- **运营**：bootstrap 接口”自锁”后，所有后续提权必须走 admin promote → **留审计痕迹**。
- **商家（路线图）**：未来 merchant 自助入驻后，商家身份独立于 admin / user，需要单独 group 和 DAO 层 shop_id 隔离。
- **客服**：业务码 30005 ”需要管理员权限”出现时，客服要能识别这是**正常 RBAC 拦截**而不是 bug。

冷启动有解、提权可追、降权快、误打有交代——这是 admin 体系的四个业务目标。

- 上线一个全新 gomall 实例，users 表是空的。
- 想给”运维大佬”开 admin 权限，必须调 /admin/users/promote。
- 但 /admin/... 子组的中间件链是 Auth + RequireRole("admin")。
- 此时一个 **admin** 都没有，谁也进不去 **admin** 子组 → 没人能把别人提升为 admin。

这就是 admin bootstrap 必须存在的原因：先有第一个 **admin**，后面 **RBAC** 才能转起来。

internal/admin/routes.go:10 admin/bootstrap 仅挂 authed, 无 RequireRole; internal/admin/service.go:65-79 BootstrapPromoteSelf



- **核心校验：** `BootstrapPromoteSelf` 在执行前 `SELECT count(*) WHERE role='admin'`。
- `count = 0`：把当前调用者升为 `admin`。
- `count > 0`：返回错误“系统已存在 `admin`”，**接口从此永久不可用**。
- 路由层做法：`authed.POST("admin/bootstrap")`，仅在 `AuthMiddleware` 之下，但不挂 `RequireRole`。

internal/admin/service.go:71-77 count 校验; internal/admin/routes.go:10

代码片段 3: BootstrapPromoteSelf 一次性校验

Listing 2: *internal/admin/service.go:65-79 BootstrapPromoteSelf*

```
65 func (s *AdminSrv) BootstrapPromoteSelf(ctx context.Context) error {
66     u, err := ctl.GetUserInfo(ctx)
67     if err != nil { return err }
68     db := user.NewUserDao(ctx).DB
69     var count int64
70     if err := db.Model(&user.User{}).
71         Where("role = ?", user.RoleAdmin).
72         Count(&count).Error; err != nil {
73         return err
74     }
75     if count > 0 {
76         return errors.New("系统已存在 admin, 禁止使用 bootstrap 接口")
77     }
78     return s.PromoteToAdmin(ctx, u.Id)
79 }
```

逐行讲解 3: BootstrapPromoteSelf

- `ctl.GetUserInfo` 取的是 `AuthMiddleware` 写进 `ctx` 的 `user id` ——调用者必须已登录。匿名打 `bootstrap` 会先被 `AuthMiddleware` 截掉。
- `count(*) WHERE role='admin':` 业务自检。`users.role` 有 `index`，实际是 `range scan`，毫秒级。
- `count > 0 return err:` 这一行就是接口“自锁”机制。
- `PromoteToAdmin:` 复用提权逻辑，触发 `InvalidateRoleCache` 立即生效。
- 潜在并发坑：两人同时打 `bootstrap`，`count=0` 都通过 → 两人都被提为 `admin`。第一次部署窗口通常只有运维一人，可接受。真要修：`SELECT ... FOR UPDATE` 或表级 `advisory lock`。

admin bootstrap ” 占用” 的运维 SOP（客服 + SRE 协同）

情境：运维 B 在 prod 上第二次调 /admin/bootstrap，得到错误” 系统已存在 admin”。

角色	该做什么
运维 B（调用者）	不要重试，先排查 admin 是不是已有人开过
客服话术	不直面用户——这是内部 SOP，对外无客诉
SRE 操作	查 admin_audit（路线图）或 <code>SELECT id, name FROM users WHERE role='admin'</code> 找到首位 admin
后续路径	让首位 admin 走 /admin/users/promote 把 B 提为 admin
真实事故钩子	若首位 admin 离职 → 走数据库直改 role（堡垒机审计），不再走 bootstrap

- bootstrap” 自锁” 的意义：排除” 调两遍把别人覆盖掉” 的灾难。
- 没有错误码（不是 30005），是因为这不该被 C 端看到，只可能出现在 SRE 控制台。

C 端 / merchant / admin 三层对比

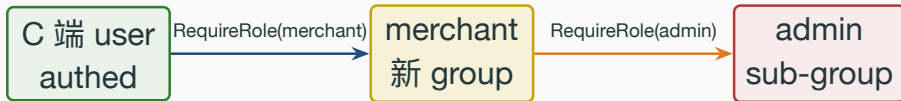
角色	路由位置	中间件链	业务能力
匿名 user	v1 顶层	仅全局桶	浏览商品
C 端 user	authed 组	+ AuthMiddleware	下单 / 抢券
merchant	暂无	—	路线图
admin	admin 子组	+ RequireRole	提权 / 后台

- C 端 vs admin 边界靠 `RequireRole("admin")` 拦下。
- merchant 还没独立 group —— 商家身份只是 user 表 Role 字段的占位。
- `product/create` 现在在 authed 组：任意已登录用户都能调，**没有商家校验**。

routes/router.go:52–53 authed; internal/product/routes.go:20–22 product/create 待迁;

internal/admin/routes.go:13–15 admin

merchant group 的路线图：路由 + DAO 双层



1. authed 内增设 merchant := authed.Group("/merchant"), 挂 RequireRole("merchant","admin")。
2. 商品 CRUD / 库存 / 订单导出全部迁过来。
3. users 表加 shop_id 外键, DAO 层每个 query 强制带 shop_id = current_user.shop_id。

RBAC 解决”他能不能进这个路由”；shop_id 解决”他能不能动这条记录”。两者缺一不可。

业务场景：补券 / 业务码 / 客服话术

本节角色视角：客服 + 运营（主角）

- 前 3 个 section 把”系统怎么做”讲完了。这一节翻面——用户怒投诉时客服该说什么。
- 鉴权出错的用户视角永远只有 5 个字：”为什么不行”。客服话术必须把 30001 / 30002 / 30005 翻译成用户能听懂、又不暴露系统细节的人话。
- 业务码的设计原则：**对外少**——用户看到的提示越统一越好，防枚举；**对内多**——客服 / SRE 拿到的码越细越好，便于审计和定位。

业务码不是给程序员看的，是**给客服 SOP 用的**。一个码对应一句话术 + 一个排查路径。

客服话术总表：用户看到什么 / 客服说什么 / 运维做什么

码	用户看到	客服该说	运维该做
400	"请重试"	"升级 app 再试"	查 SDK 版本分布，可能 OS 推送
10003	用户名或密码错	一致话术（防枚举）	看登录失败计数曲线
10005	用户名或密码错	一致话术（防枚举）	失败激增触发撞库告警
10011（路线图）	弹人机验证	"完成验证后重试"	看验证码通过率
30001	"请重新登录"	"请重新登录"	检查 token 是否被截断
30002	"10 天未登录"	"您已超 10 天未登录，请重登"	一般不需处理，若集中爆发查 jwt secret 轮换
30005	"需要管理员权限"	"该操作需要管理员，请联系您的客户经理"	查用户是不是误打 admin URL

- **30001 vs 30002** 给客服：用户视角都是“重登”，但客服比例曲线异常 → 风控介入（30001 突增多半是被换设备登录）。
- **10003 vs 10005** 给用户：必须一致——否则黑产能用“用户名不存在”vs“密码错”差异枚举用户名。

`pkg/e/code.go:8 InvalidParams=400; :13-15 10003/10005; :35-39 30001-30005`

客服补券的路径选择

方案	路径	黑产风险	审计
A. 用户自助补	C 端调 /coupon/claim	高	无
B. 客服走 admin	admin 后台接口	低	有

- **A 不行**：用户自助意味着 token 持有者就能领，黑产万号脚本 1 分钟领光所有券。
- **B 是标准**：补券必须由有 **admin** 权限的客服发起，每次请求都留 **operator_id** 进审计日志。
- gomall 当前 admin 子组只有 promote / backfill，补券接口待补。
- 路线图：admin.POST("coupon/grant", ...) 接收 target_user_id + reason，写 admin_audit 表。

internal/admin/routes.go:13-15 admin 子组当前只有 3 个接口；deck 10 SlidingWindow 防自助补券黑产

业务码 + 客服话术（鉴权三种姿势）

码	含义	客服话术
400 InvalidParams	token 没带 / 格式错	” 请更新最新版 app 再试”
30001 ErrorAuthCheckTokenFail	token 解析失败	” 请重新登录”
30002 ErrorAuthCheckTokenTimeout	双 token 都过期	” 您已超过 10 天未登录，请重新登录”
30005 ErrorAuthInsufficientAuthority	角色不够	” 该操作需要管理员权限”

- 400 vs 30001：客服区分”**SDK bug** vs **token 真坏**”。
- 30001 vs 30002：用户视角都是”要重登”，但客服能区分”账户安全事件 vs 正常长闲置”。如果某用户 30001/30002 比例异常，可能是被换设备登录，触发安全审计。
- 30005：admin 接口必备——普通用户摸 admin 时给清晰提示。

鉴权路线图（按优先级）

1. **P0**: 登出 / 改密码 token 黑名单 (Redis set + TTL)。
2. **P0**: admin 操作审计表 admin_audit (operator / target / reason / ts)。
3. **P1**: merchant 独立 group + DAO 层 shop_id 强制隔离。
4. **P1**: bootstrap 接口加 advisory lock, 防并发提权。
5. **P2**: admin 子组的”补券 / 退款 / 封号”标准接口。
6. **P2**: access 续期改”剩余 < 1h 才续”, 省一次 HS256。

鉴权不是”加一个 if”, 是用最少的代码代价表达最严的业务边界。

middleware/jwt.go: 待加黑名单; *internal/admin/service.go*: 待加 advisory lock; *routes/router.go*: 待加 merchant group

攻击面：登录接口要扛三种黑产打法

攻击类型	特征	单点拦截手段
撞库 (credential stuffing)	泄露库扫所有用户	失败限频 + 二次验证
密码喷洒 (spraying)	弱密码扫万人	全局账号失败计数
账号枚举	错误码差异爆破	错误码归一 / 时序对齐

- gomall 当前登录仅 bcrypt + 业务码 10005，无失败限频。
- 错误码上 10003（用户不存在）vs 10005（密码错）可被批量枚举。
- 黑产场景：3000 IP 代理池 × 5 RPS = **15k RPS**，bcrypt 直接拖死。

`internal/user/service.go:82-118 login` 当前无失败限频；`pkg/e/code.go:10003 vs 10005` 错误码差异是枚举漏洞

撞库防御三段式：限频 + 验证码 + 设备指纹



- 第一层：IP 维度滑动窗口限频，复用 deck 10 SlidingWindow。
- 第二层：账号维度连续失败计数，5 次失败强制带验证码。
- 第三层：15min 内累计 10 次失败 → 冻结 30min，直接返回 10005 不消耗 bcrypt。
- bcrypt cost=12 每次 250ms，账号冻结后省的是 **CPU** 不是用户体验。

`repository/cache/ratelimit.go:45 SlidingWindowAllow` 可直接复用；`internal/user/service.go:82` 是接入点

代码片段 4：登录失败计数 + 强制验证码门槛

```
1  const (  
2      loginFailKeyPrefix = "auth:login:fail:"  
3      loginFailThreshold = 5 // 15min 内  
4  )  
5  
6  func (s *UserSrv) preflight(ctx context.Context, name, cap string) error {  
7      fails, _ := cache.RedisClient.Get(ctx, loginFailKeyPrefix+name).Int()  
8      if fails >= loginFailThreshold && cap == "" {  
9          return ErrCaptchaRequired // 业务码 10011  
10     }  
11     if fails >= loginFailThreshold {  
12         if ok, _ := captchaVerify(ctx, cap); !ok { return ErrCaptchaInvalid }  
13     }  
14     return nil  
15 }
```

路线图: *internal/user/service.go login* 入口前置校验

逐行讲解 4: preflight + 失败计数器

- loginFailKeyPrefix: key 设计成 auth:login:fail:<username>, TTL = 15min 滑动窗口。
- Get(...).Int(): Redis miss 直接返回 0, 无需额外判空。
- fails $\geq$ 5 && captcha == "": 前端拿到 **10011** 直接弹 hCaptcha。
- 登录成功后必须 Del(loginFailKeyPrefix+name), 否则用户输错一次就锁 15min。
- 设备指纹路线图: UA + canvas hash + 时区组成 deviceId, 跨账号失败 $\geq$ 3 直接拉黑。

repository/cache/common.go:14 cache.RedisClient 已就绪; deck 10 SlidingWindow 提供窗口算法

双 token 失效 / 强制下线 / 多端隔离

本节角色视角：客服 + 风控 + 商家（路线图）

- **客服**：用户哭诉“账号被盗了” → 客服必须 5 分钟内**强制下线所有端**，不能等 access 24h 自然过期。
- **风控**：同 userId 同时出现在 2 个 IP 段 → 自动触发强制下线。这一步是**机器决策**、客服只通知。
- **商家（路线图）**：商家账号被盗影响整店 GMV，强制下线 SLA 必须 $< 1\text{min}$ (vs C 端 5min)。
- **业务后果 ****：盗号场景如果 24h 才失效，平均一个被盗号造成的退款 / 客诉成本 $\sim 200\text{--}500$ 元。强制下线把这个数压到 < 20 元。

失效慢是 JWT 的“原罪”。token_ver 是用一字段成本 + 一次 DB UPDATE 把这罪赎回来。

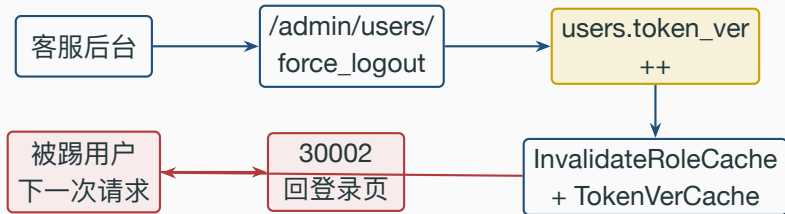
token 黑名单方案：版本号 vs SET

方案	实现	代价
A. Redis SET 黑名单	SISMEMBER jti	每请求 1 RTT, TTL 难管
B. 用户 token 版本号	users.token_ver, 签进 claims	每请求查 DB / 本地缓存
C. 用户最早签发时间戳	users.token_min_iat, 对比 iat	改密码只写一次

- A 按 token 粒度 (被偷场景), jti 数随活跃用户线性增长。
- B/C 按用户粒度 (改密码 / 强制下线), 改一次字段全部作废。
- gomall 推荐 **B+C 合并**: 版本号 ++ 即生效, 本地 sync.Map 命中 0ms。

internal/user/model.go:19-30 User struct 待加 TokenVer; pkg/utils/jwt/jwt.go:18-22 Claims 加字段

客服强制下线时序



- 写 token_ver: 单条 UPDATE 行锁, 毫秒级返回。
- 失效本地缓存: 与 RBAC 共用 InvalidateRoleCache 模式, 按 userId 删本地 sync.Map 槽。
- 多实例场景: 写 Redis pub/sub 让所有实例 Del 本地缓存槽, 最终一致延迟 < **100ms**。

internal/admin/service.go:46-61 PromoteToAdmin 已是模板; 路线图: *internal/admin/service.go* 加

ForceLogout(userId)

- 识别：同 userId 的 refresh 出现在 2 个不同 IP / device → 钓鱼信号。
- 触发：每次登录 / 刷新写 userId → deviceId HSET, device 数 > 5 触发风控。
- 处置：token_ver++ 全端下线, C 端跳登录页 + 短信 / 邮件二次验证。
- 兜底：每次发 access 时推 security_event 到 IM / 邮件, 让用户主动报警。

路线图: repository/cache/security 新增 device 集合; jwt.go SetToken 后写一次

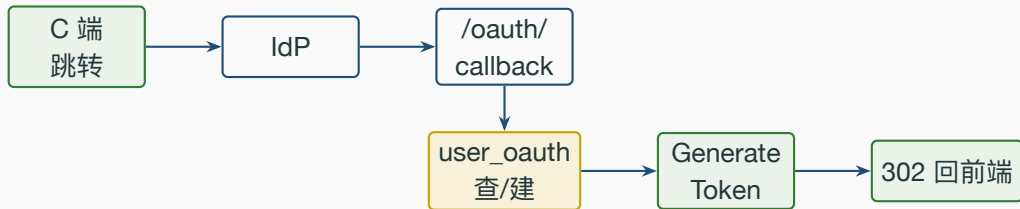
第三方登录接入对照

渠道	协议	关键字段	业务难点
微信 (开放平台 / 小程序)	OAuth2 变体	unionid / openid	账号合并
Google Sign-In	OIDC	sub / email	邮箱冲突
Apple Login	OIDC	sub / relay 邮箱	relay 去重
GitHub / 企业 SSO	OAuth2/SAML	login / orgs	角色映射

- gomall 当前只有用户名密码，第三方登录从 0 接入。
- 数据模型加 user_oauth 表：(provider, external_id) → user_id。
- **邮箱已注册场景**：合规要求二次验证，不直接合并。

路线图：internal/user/model_oauth.go；internal/user/oauth.go；router.go 加 /oauth/callback

第三方登录链路：把 OAuth 收敛回 gomall 双 token



- OAuth 拿 external_id 后不直接颁 token，先查 / 建 user_oauth。
- 后续接口仍走双 token + AuthMiddleware，IdP 不参与每次鉴权。
- external_id 已绑别人 → 拒绝，返回 ErrorExistUser。

pkg/utils/jwt/jwt.go:25-52 GenerateToken 复用；路线图：internal/user/oauth.go BindOrCreate

bcrypt cost 调优：为什么是 12 不是 10 也不是 14

cost	单次耗时 (M1)	1 核 / 秒	1k 登录耗 CPU
10	~60ms	16 次	60s
12	~250ms	4 次	250s
14	~1.0s	1 次	1000s

- cost=10 太弱：消费级 GPU 一秒穷举几千候选，2026 标准 ≥ 12 。
- cost=14 太重：登录多 1s，1k QPS 吃满整机 CPU。
- cost=12：250ms 在 300ms 感知阈值内；穷举 10 位数字 ≈ 80 年。
- bcrypt 内置 16 字节 salt，拖库后 rainbow table 失效。每 2-3 年 cost++ 一次。

internal/user/model.go:33 PasswordCost=12; :40-53 SetPassword/CheckPassword

代码片段 5: cost=12 与未来无痛升级路径

```
33  const PassWordCost = 12
34
35  func (u *User) SetPassword(pwd string) error {
36      b, err := bcrypt.GenerateFromPassword([]byte(pwd), PassWordCost)
37      if err != nil { return err }
38      u.PasswordDigest = string(b); return nil
39  }
40
41  func (u *User) CheckPassword(pwd string) bool {
42      if bcrypt.CompareHashAndPassword(
43          []byte(u.PasswordDigest), []byte(pwd)) != nil { return false }
44      if cost, _ := bcrypt.Cost([]byte(u.PasswordDigest)); cost < PassWordCost {
45          _ = u.SetPassword(pwd); _ = saveDigest(u) // 顺便升 cost
46      }
47      return true
48  }
```

internal/user/model.go:33-53 + 升级钩子

为什么 gomall 选纯 JWT，不走 session cookie

维度	Server Session	JWT (双 token)
鉴权 RT	读 Redis 1 RTT	解 HS256 零 RTT
失效粒度	删 session $O(1)$	黑名单 / token_ver
水平扩缩	共享存储瓶颈	无状态，加机器即可
跨域 / 多端	cookie SameSite	header 跨域无障碍

- gomall 优先：鉴权延迟 + 水平扩展 + 多端 → JWT。
- 代价：失效慢，用 token_ver + 黑名单兜底关键路径。
- 反例：银行类业务选 session 更合理，失效粒度比延迟重要。

`middleware/jwt.go` 全文 81 行无 Redis 依赖；session 方案需要每请求 `Get(sid)`

多端同账号：app / web / 小程序 token 隔离

- 当前 gomall 所有端共享同一对 token：app 登录 web 自动登录。
- 业务影响：web 改密码 → app 上 token 仍能用 24h。
- 路线图：Claims 加 client_type (app / web / mp / pad)，按 (userId, client_type) 颁发独立 token。
- 副作用：refresh 必须保留 client_type；强制下线粒度更精细；device 数风控按 client_type 分桶。

pkg/utils/jwt/jwt.go:18-22 Claims 待加 ClientType; internal/user/service.go:82-118 login 待区分入口

审计日志: admin 操作必须留痕

- 当前 admin 3 个接口 (promote / backfill / users), **无审计写入**。
- 合规字段: operator_id / target_id / action / reason / before / after / ts / ip / UA。
- 路线图: admin_audit 表 + AuditMiddleware 挂 admin 子组自动入表。
- 索引: (operator_id, ts) 客服自查; (target_id, ts) 投诉反查。
- 不允许 DELETE, 按月分区, 异地冷备 7 年。审计是补券 / 强制下线的前置依赖。

routes/router.go:54-55 admin 子组待加 AuditMiddleware; internal/admin/service.go 所有写操作待加审计

鉴权链路涉及的全部业务码 + 客服对照

码	含义	客服话术
400 InvalidParams	token 缺失 / 格式错	升级 app 重试
10003 ErrorNotExistUser	用户名不存在	用户名或密码不正确
10005 ErrorNotComparePassword	密码错	用户名或密码不正确
10011 ErrCaptchaRequired (路线图)	失败过阈值	请完成人机验证
30001 ErrorAuthCheckTokenFail	token 解析失败	请重新登录
30002 ErrorAuthCheckTokenTimeout	双 token 都过期	已超 10 天未登录
30003 ErrorAuthToken	token 字段不匹配	内部错误, 请重试
30005 ErrorAuthInsufficientAuthority	角色不够	需要管理员权限
SOP: 10003 / 10005 对外话术一致 (防枚举), 对内保留具体码用于审计。		

pkg/e/code.go:10001-10011, 30001-30010; 路线图: 10011 新增

鉴权链路容量规划：1k / 10k / 100k QPS 怎么撑

目标 QPS	access 解析 CPU	RBAC DB QPS (miss)	Redis RTT
1k	单核 12%	≈ 33	0.3ms p99
10k	4 核 30%	≈ 330	0.5ms p99
100k	32 核 60%	≈ 3.3k	1.5ms p99

- 压测基线：authed 链路 **58,406 RPS / p95 5.0ms**，单机已扛 50k+。
- 10k QPS 内单实例即可；100k 横向扩 4 实例 + 本地缓存命中 > 99%。
- 瓶颈不在鉴权：下单 / 支付 / 库存先到瓶颈，鉴权剩 60%+ 余量。

stressTest/REPORT.md:34-46 RPS 表行 1 (Ping 64254) / 行 3 (authed 58406)

面试 Q&A (上)

Q1: access 24h / refresh 10d 这两个数字怎么定?

A: access 短限泄露损失, refresh 长覆盖周末回流, 10d 是留存与安全的折中。

Q2: RBAC 30s 内存缓存, admin 误授权 30s 不是漏洞吗?

A: 常规延迟可接受。关键路径 InvalidateRoleCache(userId) 显式失效, 提权 / 降权立即生效。

Q3: 登出后老 token 还能用 24h 怎么办?

A: token 仅控访问不直接控钱, 大额二次验证。路线图加 Redis 黑名单 + token_ver。

Q4: bcrypt cost=12 怎么选? 怎么无痛升 14?

A: 250ms 在 300ms 感知阈值内, 穷举不现实。登录成功路径探测旧 hash 顺便 re-hash, 90 天自然升完。

Q5: 登录接口怎么扛 15k RPS 撞库?

A: IP + 账号双维度滑动窗口 + 5 次失败强制验证码 + 30min 冻结, bcrypt 不消耗在冻结账号上。

Q6: admin bootstrap 一次性接口怎么防重?

A: count(*) WHERE role='admin', > 0 直接返回错。并发加 advisory lock 兜底。

面试 Q&A (下)

Q7: merchant 没独立 group 为什么不阻塞上线?

A: 单店家场景 user 即 merchant。多店家才需 group + DAO 层 shop_id 双层。

Q8: 业务码 30001 vs 30002 用户视角一样, 区分有何意义?

A: 用户都重登, 客服区分”安全事件 vs 长闲置”, 比例异常触发审计。

Q9: 客服强制下线如何实现? 多端怎么隔离?

A: users 加 token_ver 签进 claims, 改密码 / 下线时 ++。多端按 client_type 分桶存 token_ver_map。

Q10: 为什么 gomall 选 JWT 不选 session?

A: 零 RTT + 无状态扩展 + 多端 header 无跨域。代价是失效慢, token_ver 兜底。银行业务选 session 更合理。

Q11: 第三方登录怎么接入?

A: OAuth 拿 external_id → 查/建 user_oauth → 复用 GenerateToken。后续仍走 AuthMiddleware。

Q12: 100k QPS 鉴权怎么扩? 瓶颈在哪?

A: HS256 + sync.Map 单机 58k RPS, 100k 扩 4 实例够用, 瓶颈始终在下单 / 支付 / 库存。

代码位置一览

路由 / authed / admin *routes/router.go:46–55 组合根; internal/user/routes.go:10–11;
internal/admin/routes.go:10, 13–15*

AuthMiddleware / SetToken *middleware/jwt.go:16–64, 66–73*

双 token 续期 / EmailClaims *pkg/utils/jwt/jwt.go:25–52, 67–93, 95–134*

RequireRole + 30s 缓存 / Invalidate *middleware/rbac.go:22–45, 48–86, 88–91*

admin bootstrap / promote *internal/admin/service.go:46–61, 65–79*

bcrypt cost=12 / Role *internal/user/model.go:28, 33, 40–53*

业务码 / 时长常量 *pkg/e/code.go:35–58; consts/user.go:31–32*

滑动窗口 / Redis 客户端 *repository/cache/ratelimit.go:45 SlidingWindowAllow;
repository/cache/common.go:14*

压测数据来源 *stressTest/REPORT.md:34–46*

路线图 *internal/user/service.go 加 TokenVer / TokenVerMap; internal/admin/service.go 加
ForceLogout + AuditMiddleware; internal/user/oauth.go 新建*